



TALLER PROYECTOS _ UNIDAD 3: Programación

Estudiantes:

Nombres y apellidos completos

Nombres y apellidos completos

Taller grupal (máximo de 3 estudiantes)

Estructura del Taller (2 horas aprox.):

1. Introducción (10 minutos):
 - Bienvenida y presentación del taller.
 - Presentación del desafío.
3. Actividad 1: Caza de Errores (60 minutos)
4. Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos)

Distribución de Puntajes:

- Actividad 1: 2.5 puntos
- Actividad 2: 2.5 puntos
- Total: 5 puntos



Actividad 1: Caza de Errores (60 minutos) 🕵️

- Descripción: Los estudiantes recibirán un proyecto Python simplificado desde un repositorio de control de versiones. Deberán usar las herramientas del IDE para identificar y corregir errores intencionales (errores de sintaxis, lógicos, etc.).
- Recursos:
 - Repositorio de código fuente con errores intencionales:
<https://github.com/TheAlgorithms/Python>. Específicamente, revisa el archivo "inverse_of_matrix.py" que se encuentra dentro de la carpeta "matrix" para encontrar errores.
 - Lista de errores comunes (Ver material complementario)
- Criterios de Evaluación:
 - Identificación y corrección de errores (2.5 puntos)
 - Identifica y corrige todos los errores de sintaxis (1 punto)
 - Identifica y corrige todos los errores lógicos (1 puntos)
 - Documenta las correcciones realizadas (0.5 puntos)

Nota: En caso de encontrar un error diferente a un error de sintaxis o de lógica también documéntalo.

Para esta actividad anexa el código de los errores corregidos en este apartado y identifica el tipo de error

Sube el archivo .py del proyecto corregido junto con el archivo Word de este taller.

- Puntaje Total: 2.5 puntos



Actividad 2: Ampliando Horizontes con Integración Continua (90 minutos) 🚀

- Descripción: Los estudiantes implementarán una nueva funcionalidad en el proyecto, utilizando las herramientas del IDE para la gestión de versiones y la integración continua. Deberán crear una rama (branch), desarrollar la funcionalidad, realizar pruebas y fusionar (merge) los cambios al código principal.
- Recursos:
 - Documentación de la nueva funcionalidad a implementar.
 - Guía básica sobre gestión de versiones e integración continua.

Repositorio Python actividad 2

- Actividad 2 (Ampliando Horizontes con Integración Continua):
 - [Flask Taskr](#): Este repositorio contiene una aplicación Flask sencilla con pruebas unitarias. ¡Perfecto para practicar la implementación de nuevas funcionalidades y la gestión de versiones con integración continua! 🚀

Guía Básica de Gestión de Versiones e Integración Continua:

- ¿Qué es Git? Explica que Git es un sistema de control de versiones que permite rastrear los cambios en el código. Es como un "historial" de todas las modificaciones. 📄
- ¿Qué es un repositorio? Un lugar donde se guarda el código del proyecto (como el "Flask Taskr" que mencionaste). 📁
- ¿Qué es una rama (branch)? Una copia del código principal donde se pueden hacer cambios sin afectar la versión original. ¡Ideal para probar cosas nuevas! 🌱
- ¿Qué es un "commit"? Una "foto" de los cambios realizados en el código en un momento dado. Cada "commit" debe tener un mensaje descriptivo explicando qué se modificó. 📷
 - Un "commit" es como guardar una versión del archivo con una nota. Por ejemplo:
 - "feat: Agrega función para marcar tarea como urgente"



- "fix: Corrige error al mostrar la fecha de vencimiento"
- ¿Qué es un "merge"? La acción de combinar los cambios de una rama de nuevo al código principal. 🤝
- ¿Qué es Integración Continua (CI)? La práctica de integrar (fusionar) los cambios de código de todos los desarrolladores en un repositorio compartido, de forma frecuente. Esto permite detectar errores rápidamente y mantener el código estable. 🔄

Nueva Funcionalidad a Implementar: Priorización de Tareas con Etiquetas 🏷️

Descripción: Los estudiantes deberán añadir la capacidad de asignar etiquetas de prioridad a las tareas en la aplicación Flask Taskr. Habrá tres niveles de prioridad: "Alta", "Media" y "Baja". Estas etiquetas se mostrarán junto a cada tarea en la lista de tareas.

Detalles de la Implementación:

1. Modificar el Modelo de Datos:

- Añadir un campo llamado priority al modelo de datos de la tarea. Este campo deberá ser una cadena de texto (String) y podrá tener uno de los tres valores: "Alta", "Media" o "Baja".
- Establecer un valor por defecto para la prioridad (por ejemplo, "Media").

2. Modificar el Formulario de Creación/Edición de Tareas:

- Añadir un campo de selección (SelectField) al formulario para que el usuario pueda elegir la prioridad de la tarea al crearla o editarla.
- Las opciones del campo de selección deberán ser "Alta", "Media" y "Baja".

3. Modificar la Vista (Template) de la Lista de Tareas:

- Mostrar la etiqueta de prioridad junto a cada tarea en la lista.
- Opcionalmente, se pueden usar colores diferentes para cada nivel de prioridad (por ejemplo, rojo para "Alta", amarillo para "Media" y verde para "Baja").

4. añadir Estilos (CSS) Opcionales:

- Si se desea, se pueden añadir estilos CSS para que las etiquetas de prioridad se vean más llamativas.



Flujo de Trabajo:

1. Clonar el repositorio de Flask Taskr.
2. Crear una rama llamada prioridad-tareas.
3. Modificar el modelo de datos, el formulario y la vista (template) según las indicaciones anteriores.
4. Realizar "commits" con mensajes descriptivos (ej: "feat: Añade campo de prioridad al modelo de tarea", "feat: Agrega selector de prioridad al formulario", "feat: Muestra la prioridad en la lista de tareas").
5. Subir la rama al repositorio.
6. Crear una "Pull Request" para fusionar los cambios. Ver video explicativo de como realizarlo: <https://www.youtube.com/watch?v=Zqft6yNRuNs>

Para esta actividad anexar el enlace del Pull Request creado y el código con las modificaciones anteriores en este apartado.

Criterios de Evaluación Específicos:

- Implementación del Modelo de Datos: ¿Se ha añadido el campo priority correctamente al modelo de datos? (0.25 puntos)
- Implementación del Formulario: ¿Se ha añadido el campo de selección de prioridad al formulario? (0.25 puntos)
- Visualización en la Vista: ¿Se muestra la etiqueta de prioridad correctamente en la lista de tareas? (0.25 puntos)
- Uso de "Commits" Descriptivos: ¿Se han realizado "commits" con mensajes claros y descriptivos? (0.25 puntos)
- Creación del Pull Request (1.5)